

# CERNAL Computational Pipeline Map

## Table of contents:

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>CERNAL.....</b>                                      | <b>2</b>  |
| <b>Document's Purpose.....</b>                          | <b>2</b>  |
| <b>Flow Chart.....</b>                                  | <b>2</b>  |
| <b>General Considerations.....</b>                      | <b>2</b>  |
| <b>Components Description.....</b>                      | <b>3</b>  |
| User's Inputs.....                                      | 3         |
| Supporting Databases.....                               | 4         |
| Gene Selection.....                                     | 4         |
| Initial Trigger Scoring.....                            | 5         |
| Switches Design.....                                    | 5         |
| Circuit Design.....                                     | 6         |
| Plasmid Construction.....                               | 6         |
| Compiler's Output:.....                                 | 7         |
| <b>General Shared Functions.....</b>                    | <b>7</b>  |
| <b>Appendix A: Components Table.....</b>                | <b>9</b>  |
| <b>Appendix B: Duty Assignment.....</b>                 | <b>10</b> |
| <b>Appendix C: Guiding Principles and Theories.....</b> | <b>11</b> |
| <b>Appendix D: Additional Information.....</b>          | <b>12</b> |

## CERNAL

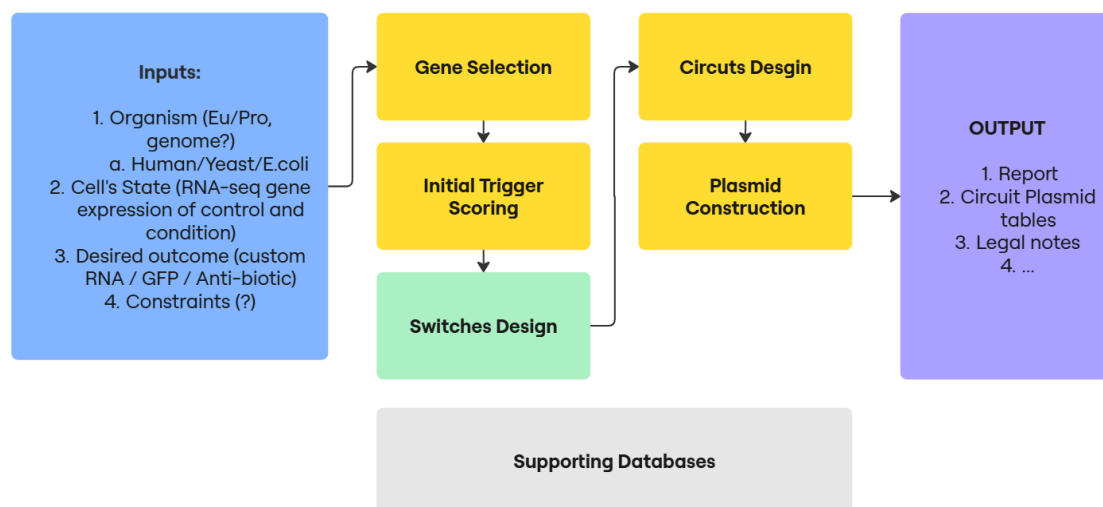
Compiler-like Engine for RNA Logic, designs RNA logic circuits on a plasmid from transcriptomic signature.

## Document's Purpose

Organizing iGEM's 2026 TAU computational team efforts to build the CERNAL:

1. Mapping the computational pipeline of CERNAL to its modules.
2. For each module, define:
  - a. General / independent module.
  - b. Inputs / Outputs.
  - c. Assigned person - TBD.
  - d. Implementation guidelines description.

## Flow Chart



**Legend:** Inputs, Supporting Databases, General Components, Specific Component, Output

**Fig.1 CERNAL Flow Chart**

## General Considerations

1. "Comparability" - input and output units, normalization etc.
  - a. Equilibrium Energy metrics will be used. Future examination of kinetic metrics.
2. Step to step interactions - defined format of inputs and outputs.
  - a. Mainly .csv files with id numbers for each part for traceability.
3. Computational and data size limitations.

- a. Applied filters and output data tables columns.
- b. **CandidateStore** owns all persisted inter-stage data (CSV + provenance); every stage reads its input and writes its output through it — no stage passes objects directly to a non-adjacent stage. Shared utilities are **stateless** (pure functions over sequences/structures), which is what makes caching and parallel fan-out safe.
- 4. iGEM's judgebook guidelines:
  - a. Forward and reverse engineering - CERNAL is a forward engineering tool.
    - i. How and should we think of a calibration pipeline to implement reverse engineering?
    - ii. Should we implement a design cycle loop ? (e.g. relevant suggestions in the pdf report, second optimization iteration with wet lab feedback results?)
  - b. Robustness and Reproducibility -
    - i. The outputs and code should be well described and friendly.
    - ii. How can a user use it for a different organism?
  - c. Clearly defined quantifiable criteria/measures of success - requires reflective scoring (choosing a good proxy to predict success).
    - i. reliable, important, reproducible, generic, robust...

## Components Description

### User's Inputs

1. Organism selection: "Human" / "E.coli" / "Yeast"
2. RNA-seq normalized/raw counts matrix as .csv.

| GeneId | Symbol | Sample1 | ... | SampleN |
|--------|--------|---------|-----|---------|
| b0002  | thrA   | 2272    | ... | 2273    |

- Including meta data determining the control and condition columns/samples.

3. RNA-seq DGE results as .csv

| Gene Id | Base Mean | Log 2 Fold Change | Lfc SE | stat  | P Value | P Adj | Symbol |
|---------|-----------|-------------------|--------|-------|---------|-------|--------|
| b0002   | 2272.37   | -0.29             | 0.37   | -0.79 | 0.42    | 0.53  | thrA   |

- **Gene Id**: The unique ID number or name for a gene in the database.

- Base Mean: The average expression value of the gene across all samples.
  - **Log 2 Fold Change**: The size of the change in expression (in log2 scale). A higher number means a bigger change between groups.
  - Lfc SE: The standard error of the log2 fold change estimate.
  - stat: The statistical test value used to calculate significance.
  - P Value: The raw probability that the gene changes is random.
  - **P Adj**: The adjusted p-value after fixing for false positives in multiple tests.
  - Symbol: The standard short name or common label for the gene.
4. Desired outcome selection: “GFP” / “Antibiotics” / user’s RNA sequence

*\* User’s input should go through QC*

## Supporting Databases

5. Organisms’ codon-usage table.
6. Gene sequences database – maybe depends on the .fasta files.
7. Multi-cell organism: Human Cell Atlas.
8. Pre-transcription sequence (plasmid).

## Gene Selection

Keeps the genes that actually separate the two cell states: big enough fold change, statistically significant, and expressed in a usable absolute range in both states (too low → false negatives, too high → false positives). Outputs a ranked shortlist with an up/down call per gene.

### Inputs:

9. [\[2\] RNA-seq count matrix .csv](#)
10. [\[3\] RNA-seq DGE results .csv](#)
11. [\[7\] Human Cell Atlas](#)

### Outputs:

12. Selected genes .csv list
  - Filters: absolute abundance, fold change and P adj thresholds.

- Columns: Gene Id, symbol, up/down regulated, Control Percentile of expression, Condition Percentile of expression, Condition Specificity, Log 2-fold change, Score.

### Guidelines:

- DGE: run PyDESeq2 → apply thresholds (log2fc, padj, absolute abundance) →
- emit a **DE-gene set** with binary up/down calls **and** continuous features.  
*Fan-in from many samples → one gene table.*

## Initial Trigger Scoring

Slides a window along each selected gene's transcript and **ranks every sub-segment by how unpaired it is in its local folding context** (RNAplfold), plus off-target load, GC, and forbidden motifs — per length-class, because each gate family needs a different footprint. Outputs ranked trigger candidates per gene.

### Inputs:

- [\[12\] Selected genes .csv list](#)
- [\[6\] Gene sequences database – maybe depends on the .fasta files](#)

### Output:

- Global Trigger Scoring .csv list
  - Filters: restricted motifs (NNNN)
  - Columns: Trigger Id, Gene Id, Symbol, Trigger Length, Start Index, Gene length, Trigger Openness, Trigger MFE, Trigger Accessibility (local context), OFF Target, Gene Segment Specificity, Ribosome occupancy, GC content, AUG indexes, STOP indexes, Sequence, Trigger Score.

### Guidelines:

- Sliding windows → rank subsegments by **accessibility** ([RNAplfold](#)) and **low off-target** load (or MFE)
- Desired lengths “classes” VS. Just indicating an accessible/open segment.

## Switches Design

For each (trigger segment × gate type), runs the architecture-specific generator to build a switch that is closed in the OFF state and opened by that

trigger, then validates it folds as intended and passes the hard sequence rules (1×AUG, 0×STOP, RBS in loop, no restriction sites).

### Inputs:

16. [\[12\] Selected genes .csv list](#)
17. [\[15\] Initial Global Trigger Scoring .csv list](#)
18. [\[4\] desired outcome](#)

### Databases:

19. [\[5\] Organisms' codon usage table.](#)
20. [\[6\] Gene sequences database – maybe depends on the .fasta files](#)
21. [\[8\] Pre-transcription sequence \(plasmid\)](#)

### Output:

22. Switch .csv list
  - Filters: 1XAUG, 0XStop, 1XRBS in loop only (Pro), Restricted motifs (NNNN)?, BioBrick Standard [RFC 10] or iGEM Type IIS Standard [RFC 1000] - e.g. absence of restricted sites.
  - Columns: Switch Id, Switch sequence, dot-bracket notation, Deviation from desired structure (ensemble defect?), Trigger Id, Gene Id, MFE ON state, MFE OFF state, MFE trigger, Switch (binding site) Accessibility, OFF Target (binding site), Translation Score (Codon usage), Final Switch Score.
23. Switch Meta data .csv – elaborating stats per-switch design for future data analysis. (e.g. toehold stem / loop length...). Influences the final switch score that varies by implementation. The final score also takes into account the Genes' expression and specificity.

### Guidelines:

- a. Dispatch to the track/gate-specific **Generator**.
  - i. Prokaryotic: toeholds (1 input, 2 input AND), antisense NOT.
  - ii. Eukaryotic: toeholds (1 input, 2 input AND), antisense NOT, CRISPR (1 input)
  - iii. 2 inputs can be of the same gene or trigger.
- b. Each generator gets an optional triggers list and designs switches for each one/combination.

## Circuit Design

Turns the genes' up/down pattern into a boolean function, generates the expressions reproducing it, keeps only those buildable from switches actually designed upstream, and ranks them by how cleanly they separate condition from control samples versus how complex they are.

**Inputs:**

- 24. [\[12\] Selected genes .csv list](#)
- 25. [\[15\] Initial Global Trigger Scoring .csv list](#)
- 26. [\[22+23\] Switch .csv list and meta data](#)

**Outputs:**

- 27. Circuits scored .csv list
  - Filters: BioBrick Standard [RFC 10] or iGEM Type IIS Standard [RFC 1000] - e.g. absence of restricted sites.
    - Pareto Frontier Filtering ?
  - Columns: Circuit Id, Logical expression, Components List (Gene, triggers, switches ids), True positive (condition Specificity), True Negative (circuit OFF-target), False Positive, False Negative, Final Circuit Score.

| Confusion Table    | Condition      | Control        |
|--------------------|----------------|----------------|
| <b>Circuit On</b>  | True Positive  | False Positive |
| <b>Circuit Off</b> | False Negative | True Negative  |

**Guidelines:**

- a. Synthesize multiple equivalent boolean expressions using switches, genes and triggers only from the inputs.
- b. Enumerate candidate circuit topologies.

## Plasmid Construction

Lays the winning circuit's parts onto a backbone with promoters, terminators, effector domains and the output gene, checks assembly-standard compliance, and exports an orderable sequence.

**Inputs:**

- 28. [\[12\] Selected genes .csv list](#)
- 29. [\[15\] Initial Global Trigger Scoring .csv list](#)
- 30. [\[22+23\] Switch .csv list and meta data](#)
- 31. [\[27\] Circuits scored .csv list](#)

**Output:**

- 32. Plasmid Id and sequence columns appended to the matching [25] Circuits scored .csv list.

## Guidelines:

- a. Assemble winning designs onto plasmid layout(s) — promoters, effector domains (for CRISPR: activator vs repressor), output gene(s), OR-at-final-level if requested → export GenBank/FASTA.

## Compiler's Output:

Assembles the evidence a user needs to decide whether to order a plasmid: designs, structures, confusion tables, separation margins, flags and caveats.

### 33. PDF Report:

- “Read Me”
- Abstract of the designed circuits and plasmids for the user to purchase a plasmid. (e.g. Top X / above threshold Plasmid sequences, confusion tables, flags, diagrams....)
- Legal notes
- ...

## General Shared Functions

| #  | Function                                    | What it does                                                                                                                                                                                                                | Inputs                         | Outputs                                                 | Consumed by                                                     | Proposed owner          |
|----|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|---------------------------------------------------------|-----------------------------------------------------------------|-------------------------|
| S1 | <b>LocalFoldProfile</b><br>(window folding) | The primitive behind trigger selection: slides a window along a transcript and returns per-position <b>unpaired probability</b> in local context. <code>RNAplfold -W L, u -L -u. openness(segment) = mean p_unpaired</code> | <code>seq:Seq, W, L, u</code>  | <code>p_unpaired: float[]</code>                        | Trigger Scoring, all switch validators, antisense accessibility | <i>shared-lib owner</i> |
| S2 | <b>FoldEngine</b>                           | MFE, partition function, ensemble defect, BPPM, suboptimal structures. Adapter over ViennaRNA / NUPACK / Yehuda's solver                                                                                                    | <code>seqcomplex</code>        | <code>{mfe, dot_bracket, ensemble, bppm, defect}</code> | every generator + validator                                     |                         |
| S3 | <b>HybridizationEnergy</b>                  | <code>ΔG_bind = G_complex - (G_switch + G_trigger)</code> — the formula is already in the toehold spec; implement once                                                                                                      | <code>switch:Seq</code> ,<br>, | <code>dG_bind: float</code>                             | toehold, CRISPR, antisense                                      | <i>shared-lib owner</i> |



|     |                                     |                                                                                                                                                                                                                 |                                                     |                                              |                                                  |                         |
|-----|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|----------------------------------------------|--------------------------------------------------|-------------------------|
|     |                                     |                                                                                                                                                                                                                 | <code>trigger:Seq</code>                            |                                              |                                                  |                         |
| S4  | <b>StructureMatch</b>               | Compares predicted MFE structure to the <i>desired</i> structure; returns deviation + folding probability of the target fold                                                                                    | <code>dot_bracket, target_structure</code>          | <code>{deviation, p_target_fold}</code>      | toehold, CRISPR (spec stages 3 & 6), antisense   | <i>shared-lib owner</i> |
| S5  | <b>OffTargetScan</b>                | <b>Two directions</b> (both required by the toehold spec): (a) switch cross-activated by non-cognate RNA → leak/false-positive; (b) trigger sponged by non-cognate RNA → weak ON/false-negative                 | <code>seq, transcriptome:FASTA, max_mismatch</code> | <code>{hits:List[Hit], penalty:float}</code> | Trigger Scoring, all validators, Circuit scoring | <i>shared-lib owner</i> |
| S6  | <b>SequenceUtils</b>                | revcomp, translate, in-frame STOP scan, alternative-AUG scan, GC content, homopolymer scan                                                                                                                      | <code>seq</code>                                    | derived sequence facts                       | everywhere                                       | <i>shared-lib owner</i> |
| S7  | <b>MotifScreen</b>                  | Rejects/penalises prohibited motifs: RFC10 / RFC1000 restriction sites, RBP binding motifs, RNase sites, homopolymers >5 nt                                                                                     | <code>seq, motif_sets</code>                        | <code>{violations:List}</code>               | Trigger Scoring, all generators, Plasmid         | <i>shared-lib owner</i> |
| S8  | <b>CodonTools</b>                   | Synonymous-codon substitution search + codon-usage/translation scoring. Needed by antisense CDS extension <b>and</b> toehold linker design                                                                      | <code>cds:Seq, organism, target_pairing</code>      | <code>{variants, translation_score}</code>   | toehold, antisense, Plasmid                      | <i>shared-lib owner</i> |
| S9  | <b>TranslationInitiationScore</b>   | RBS strength (prok) / Kozak match (euk), plus unstructured "ramp" check downstream of AUG                                                                                                                       | <code>seq, host</code>                              | <code>TIR:float</code>                       | toehold, antisense                               | <i>shared-lib owner</i> |
| S10 | <b>ScoreNormalizer + Aggregator</b> | Maps heterogeneous raw metrics onto one comparable scale + <b>a confidence field</b> . This is the "Comparability" bullet in General Considerations — currently unowned and load-bearing for cross-gate ranking | <code>raw_metric, metric_type</code>                | <code>{norm:[0,1], confidence}</code>        | Trigger, Switch, Circuit scoring                 | Compiler owner          |
| S11 | <b>CandidateStore / IO</b>          | CSV read/write, stable ID minting, provenance/lineage columns, caching by input hash. <b>Defines the [n] artifact registry</b>                                                                                  | <code>records</code>                                | CSV + handles                                | every stage                                      | Software team           |
| S12 | <b>FilterEngine</b>                 | Declarative per-stage filters — the doc already lists a                                                                                                                                                         | <code>records, filter_spec</code>                   | filtered records                             | every stage                                      | Software team           |

|     |                                     |                                                                                                                                        |                                    |                               |                         |                            |
|-----|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|-------------------------------|-------------------------|----------------------------|
|     |                                     | <b>Filters:</b> line per module;<br>implement the mechanism once                                                                       |                                    |                               |                         |                            |
| S13 | <b>TopKSelector / ParetoFilter</b>  | Enforces the stage budgets that close blocker B7                                                                                       | <b>records, K</b><br>or objectives | pruned<br>records             | every stage<br>boundary | Compiler<br>owner          |
| S14 | <b>StructureViz + ReportBuilder</b> | Structure/circuit figures and the output PDF                                                                                           | designs,<br>scores                 | figures,<br><b>report.pdf</b> | Report                  | Software<br>team           |
| S15 | <b>QC</b>                           | Validates the uploaded matrix before anything runs (distributions, PCA, outliers, batch structure) — currently a footnote with no home | <b>counts, metadata</b>            | <b>{plots, warnings}</b>      | Gene<br>Selection       | Gene<br>Selection<br>owner |

[Aviv's generator and validation codes](#) - Internal module Pass/fail structural & sequence validation, Contains generic thermodynamic/sequence/off-target/viz methods. **refactored into S1–S9 rather than copied per gate.**

[Another 2 generators](#)

## Appendix A: Components Table

TO DO

## Appendix B: Duty Assignment

### Components:

| Component               | Responsibility  | Status     | Files  | Notes                                                                                               |
|-------------------------|-----------------|------------|--------|-----------------------------------------------------------------------------------------------------|
| User's Inputs           | Shir (Software) | not s... ▾ | קובץ 0 |                                                                                                     |
| Supporting Databases    | Ze'ev           | In pr... ▾ | קובץ 0 | Plasmid - Ask Daniel for the sequence. He currently has for E.Coli and Human, Yeast is in progress. |
| Gene Selection          | Ze'ev           | Done ▾     | קובץ 0 | Condition specificity wasn't implemented.                                                           |
| Initial Trigger Scoring | Liran           | not s... ▾ | קובץ 0 |                                                                                                     |
| Switches Design         | Described below | In pr... ▾ | קובץ 0 |                                                                                                     |
| Circuit Design          | Offer           | not s... ▾ | קובץ 0 |                                                                                                     |
| Plasmid Construction    | Ziv             | not s... ▾ | קובץ 0 |                                                                                                     |
| Compiler's Output       | Ziv (software)  | not s... ▾ | קובץ 0 |                                                                                                     |

### Gates:

| Gate + architecture                                                                     | PROKARYOTIC   |             | EUKARYOTIC |             |              |
|-----------------------------------------------------------------------------------------|---------------|-------------|------------|-------------|--------------|
|                                                                                         | Toehold       | Antisense   | Toehold    | Antisense   | CRISPR dCas9 |
| LAYER 1 - Basic gates (2-input)                                                         |               |             |            |             |              |
| NOT  |               | Mor & Shani |            | Mor & Shani |              |
| OR   | Shani         |             | Ze'ev      |             | Michal & Tal |
| AND  | Liran & Offer |             |            |             |              |

## Appendix C: Guiding Principles and Theories

Merge into General Considerations?

List assumptions, approximations and simplifications.

## Appendix D: Additional Information

[RNAplfold](#): is a bioinformatics program in the [ViennaRNA Package](#) that calculates local base pair probabilities for long RNA sequences using a sliding window. [1]

### Overview and Function

- **Local Folding:** It focuses on base pairs within a defined physical distance (the maximum span) rather than folding the entire molecule at once. [1, 2]
- **Sliding Window:** It moves across the RNA sequence using a user-specified window size to compute opening and pairing probabilities. [1, 2]
- **Efficiency:** It saves memory and processing time compared to global partition function folding, making it ideal for large genomes or long transcripts. [1]
- **Accessibility Profiles:** It often generates local unpaired probabilities used by other tools (like RNAplex or RNAsnoop) to predict RNA-RNA interactions. [1, 2]

### Key Parameters

- `-w` sets the **window size** for the local calculation.
- `-l` sets the **maximum base pair span** (the furthest apart two paired bases can be).
- `-u` defines the length of the **unpaired region** of interest. [1, 2]

## Appendix E: Document's Review

1. **No method is defined to compute TP/FP/TN/FN.** With one condition and one control group there is no population to score against unless you define the unit of observation. Proposal: **evaluate the circuit per RNA-seq sample**. Limitation: with 3–6 samples per group most circuits will score perfectly and the matrix will not discriminate, so a **continuous separation margin must be the real ranking metric**, with the confusion matrix as a pass/fail gate (Pareto?).
2. **A single global trigger list cannot serve all four switch families.** The trigger footprint is set by the consuming architecture, and they differ substantially. One **Trigger Length** column and one window size will either over-restrict or emit segments some generators cannot use. **Fix:** make trigger length a *parameter set* — scan per length-class, tag each row with the class, and let each generator request its class. The length is not a global constant.
3. **Circuit depth and the cascading mechanism are undefined.** A toehold's output is a *protein*, not an RNA trigger — so a 2-layer circuit requires layer 1 to express an RNA that triggers layer 2 (extra latency and leak), whereas a CRISPR gate's transcriptional output cascades more naturally.
4. **No pruning budget.** "Computational and data size limitations" is listed as a consideration and never resolved. Order of magnitude:  $\sim 50$  selected genes  $\times \sim 10^3$  windows/gene  $\times \sim 10\text{--}10^2$  stochastic designs/segment  $\times$  circuit topologies. Every stage boundary needs an explicit **top-K**, and the K values need to be written down.
5. **Implementation guidelines** — currently summaries, not guidelines; no parameters, no algorithms, no acceptance criteria
6. **Two-pass dependency not represented** — circuit score depends on realized switch scores, but switches are designed from circuit needs. Fig.1 shows one forward pass
7. **iGEM judgebook questions raised, unanswered** — reverse engineering / calibration, design-cycle loop, other-organism portability.

### 8. Circuit Design (*supervisor priority*)

#### What happens in this module

Takes the up/down pattern of the selected genes, turns it into a boolean function that is true exactly in the target condition, generates the expressions that reproduce that function, keeps only

those buildable from switches that were **actually designed and validated upstream**, and ranks them by how cleanly they separate condition from control samples against how complex they are.

### Step 1 — Binarize and build the truth table

For each selected gene  $g$  define a threshold  $\tau_g$  (proposed: midpoint between control-group and condition-group means in log space; alternatives: percentile cut, or the value maximising sample separation). Per sample  $s$ :  $x_g(s) = 1$  if  $\text{expr}_g(s) \geq \tau_g$  else  $0$ . Target:  $f = 1$  on condition samples,  $0$  on control samples.

**Key property, currently unstated in the doc:** with  $n$  genes there are  $2^n$  possible input patterns but only  $n\_samples$  observed ones. The truth table is therefore **incompletely specified** — most rows are *don't-care*. This is not a defect; don't-cares are **degrees of freedom** that permit far simpler circuits. State it explicitly, because it changes which minimisation algorithm is correct.

### Step 2 — Generate candidate expressions

Do **not** enumerate all  $2^{(2^n)}$  functions. Instead:

1. Build the sum-of-products over **observed condition patterns** only.
2. Minimise as an incompletely-specified function (Quine–McCluskey / Espresso with don't-cares).
3. Apply algebraic restructuring — factoring, De Morgan, absorption — to generate *equivalent* forms with different gate counts and depths (this is the "does  $(A \text{ AND } B) \text{ AND } C$  beat  $A \text{ AND } (B \text{ AND } C)$ " question from Model Desc. 2.0).
4. Also emit *non-equivalent but sample-consistent* forms that exploit don't-cares.

### Step 3 — Filter to what is physically realizable

Keep only expressions implementable with the gate library **available for that organism**, using switches present in the input `Switch .csv`:

- Prokaryote: YES (toehold), NOT (antisense), NOR (antisense, pending spec), AND  **blocked by B4** for endogenous triggers unless the synthetic-linker variant is used.



- Eukaryote: YES (toehold, CRISPRa), AND (double-hairpin toehold), NOT (antisense, CRISPRi).
- OR: not a gate — implemented as **multiple switches expressing the same output** at the final layer.
- **Depth limit:** enforce whatever B5 resolves to. Until then, cap at depth 1 (single multi-input gate + OR-convergence) and treat deeper circuits as an explicit extension.

#### Step 4 — Score

**Corrected confusion table** (fixes B1):

|             | Condition (actual +) | Control (actual −) |
|-------------|----------------------|--------------------|
| Circuit ON  | True Positive        | False Positive     |
| Circuit OFF | False Negative       | True Negative      |

**How to compute it (fixes B2) — per-sample evaluation:**

1. For each RNA-seq sample  $s$  in the input matrix, evaluate the boolean expression on  $x_g(s)$  → predicted circuit output.
2. Compare to that sample's true label → one cell of the matrix per sample.
3. Aggregate over all samples → TP/FP/TN/FN, hence TPR and TNR.

**Honest limitation — this must be designed around:** with 3–6 samples per group, most surviving circuits will score perfectly and the confusion matrix **will not rank them**. Therefore:

- Use the confusion matrix as a **pass/fail gate**.
- Use a **continuous separation margin** as the actual ranking metric — e.g. the minimum distance of any sample from its gene's threshold  $\tau$ , propagated through the circuit. A circuit whose genes sit far from their thresholds in every sample is robust; one that barely clears is not.
- Add **bootstrap/leave-one-out over samples** to expose circuits that depend on a single sample.

**Full circuit score components:**

| Component                               | Source                | P        |
|-----------------------------------------|-----------------------|----------|
| Separation margin                       | per-sample evaluation | A        |
| Confusion matrix (gate)                 | per-sample evaluation | A        |
| Gate count + depth penalty              | topology              | A        |
| Input quality (log2fc, padj, abundance) | Gene Selection        | A        |
| <b>Realized per-gate ON:OFF</b>         | Switch scores         | <b>B</b> |
| Circuit-level off-target load           | S5 aggregated         | B        |

**Two-pass, closing G10:** Pass A ranks on cheap a-priori features *before* switches are designed; Pass B re-ranks once realized switch scores exist. This is the circular dependency the doc does not yet represent — Fig.1 needs a feedback edge.

### Step 5 — Select

Confirm the doc's own instinct: **Pareto frontier** over (dynamic range, separation margin, complexity). The objectives genuinely conflict and a single weighted sum hides the trade-off from the user. Then top-K off the frontier.

### Decisions this module still needs

1.  $\tau_g$  rule (midpoint / percentile / max-separation).
2. Max circuit depth + cascading species (**B5**).
3. Gate-count penalty weight.
4. Pareto vs weighted sum for the final cut.
5. Whether wet-lab results feed back as re-weighting (the judgebook "design cycle" question).